

第八章 线程

目录

- 多线程编程基础
- 线程的生命周期
- 线程的优先级
- 本章小结

8.1 多线程编程基础

- 本节内容
 - 线程的概念
 - Thread类
 - Runnable接口
 - 线程间的数据共享
 - 多线程的同步控制
 - 线程之间的通信
 - 后台线程

8.1.1 线程的概念

- 进程和线程的区别
- 进程
 - 一个独立程序的每一次运行称为一个进程，例如
 - 用字处理软件编辑文稿时，同时打开mp3播放程序听音乐，这两个独立的程序在同时运行，称为两个进程
 - 设置一个进程要占用相当一部分处理器时间和内存资源
 - 大多数操作系统不允许进程访问其他进程的内存空间，进程间的通信很不方便，编程模型比较复杂

8.1.1 线程的概念(续)

- 线程

- 一个程序中多段代码同时并发执行，称为多线程
- 通过多线程，一个进程表面上看同时可以执行一个以上的任务——并发
- 创建线程比创建进程开销要小得多，线程之间的协作和数据交换也比较容易
- **Java**是第一个支持内置线程操作的主流编程语言
- 多数程序设计语言支持多线程要借助于操作系统“原语(primitives)”

8.1.2 Thread类

- Thread类

- 在Java程序中创建多线程的方法之一是继承Thread类
- 封装了Java程序中一个线程对象需要拥有的属性和方法
- 从Thread类派生一个子类，并创建这个子类的对象，就可以产生一个新的线程。这个子类应该重写Thread类的run方法，在run方法中写入需要在新线程中执行的语句段。这个子类的对象需要调用start方法来启动，新线程将自动进入run方法。原线程将同时继续往下执行
- Thread类直接继承了Object类，并实现了Runnable接口。它位于java.lang包中，因而程序开头不用import任何包就可直接使用

8.1.2 Thread类(续)

——例8_1

- 在新线程中完成计算某个整数的阶乘

```
public class Ex8_1 {  
    public static void main( String [] args ) {  
        System.out.println("main thread starts");  
        FactorialThread thread=new FactorialThread(10);  
        thread.start();  
        System.out.println("main thread ends " );  
    }  
}  
  
class FactorialThread extends Thread {  
    private int num;  
    public FactorialThread( int num ) {  
        this.num=num;  
    }  
}
```

8.1.2 Thread类(续)

——例8_1运行结果

```
public void run() {  
    int i=num;  
    int result=1;  
    System.out.println("new thread started" );  
    while(i>0) {  
        result=result*i;  
        i=i-1;  
    }  
    System.out.println("The factorial of "+num+" is "+result);  
    System.out.println("new thread ends");  
}
```

- 运行结果
main thread starts
main thread ends
new thread started
The factorial of 10 is 3628800
new thread ends

8.1.2 Thread类(续)

——例8_1修改

- 结果说明

- `main`线程已经执行完后，新线程才执行完
- `main`函数调用`thread.start()`方法启动新线程后并不等待其`run`方法返回就继续运行，`thread.run`函数在一边独自运行，不影响原来的`main`函数的运行

- 源程序修改

- 如果启动新线程后希望主线程多持续一会再结束，可在`start`语句后加上让当前线程（这里当然是`main`）休息1毫秒的语句：

```
try { Thread.sleep(1); } catch(Exception e){};
```

8.1.2 Thread类(续)

——例8_1修改后运行结果

- 修改后运行结果

main thread starts

new thread started

The factorial of 10 is 3628800

new thread ends

main thread ends

- 运行结果说明

- 新线程结束后main线程才结束

8.1.2 Thread类(续)

——常用API函数

名称	说明
<code>public Thread()</code>	构造一个新的线程对象，默认名为Thread-n，n是从0开始递增的整数
<code>public Thread(Runnable target)</code>	构造一个新的线程对象，以一个实现Runnable接口的类的对象为参数。默认名为Thread-n，n是从0开始递增的整数
<code>public Thread(String name)</code>	构造一个新的线程对象，并同时指定线程名
<code>public static Thread currentThread()</code>	返回当前正在运行的线程对象
<code>public static void yield()</code>	使当前线程对象暂停，允许别的线程开始运行
<code>public static void sleep(long millis)</code>	使当前线程暂停运行指定毫秒数，但此线程并不失去已获得的锁旗标。

8.1.2 Thread类(续)

——常用API函数

<code>public void start()</code>	启动线程，JVM将调用此线程的run方法，结果是将同时运行两个线程，当前线程和执行run方法的线程
<code>public void run()</code>	Thread的子类应该重写此方法，内容应为该线程应执行的任务。
<code>public final void stop()</code>	停止线程运行，释放该线程占用的对象锁旗标。
<code>public void interrupt()</code>	打断此线程
<code>public final void join()</code>	在当前线程中加入调用join方法的线程A，直到线程A死亡才能继续执行当前线程
<code>public final void join(long millis)</code>	在当前线程中加入调用join方法的线程A，直到到达参数指定毫秒数或线程A死亡才能继续执行当前线程

8.1.2 Thread类(续)

——常用API函数

<pre>public final void setPriority(int newPriority)</pre>	设置线程优先级
<pre>public final void setDaemon(Boolean on)</pre>	设置是否为后台线程，如果当前运行线程均为后台线程则JVM停止运行。这个方法必须在start()方法前使用
<pre>public final void checkAccess()</pre>	判断当前线程是否有权力修改调用此方法的线程
<pre>public void setName(String name)</pre>	更改本线程的名称为指定参数
<pre>public final boolean isAlive()</pre>	测试线程是否处于活动状态，如果线程被启动并且没有死亡则返回true

8.1.2 Thread类(续)

——例8_2

- 创建3个新线程，每个线程睡眠一段时间（0~6秒），然后结束

```
public class Ex8_2 {  
    public static void main( String [] args ) {  
        //创建并命名每个线程  
        TestThread thread1 = new TestThread( "thread1" );  
        TestThread thread2 = new TestThread( "thread2" );  
        TestThread thread3 = new TestThread( "thread3" );  
        System.out.println( "Starting threads" );  
        thread1.start(); // 启动线程1  
        thread2.start(); // 启动线程2  
        thread3.start(); // 启动线程3  
        System.out.println( "Threads started, main ends\n" );  
    }  
}
```

8.1.2 Thread类(续)

——例8_2

```
class TestThread extends Thread {
    private int sleepTime;
    public TestThread( String name ) {
        super( name );
        sleepTime = ( int ) ( Math.random() * 6000 );
    }
    public void run() {
        try {
            System.out.println(
                getName() + " going to sleep for " + sleepTime );
            Thread.sleep( sleepTime ); //线程休眠
        }
        catch ( InterruptedException exception ) {};
        System.out.println( getName() + " finished" );
    }
}
```

8.1.2 Thread类(续)

——例8_2运行结果

- 运行结果

Starting threads

Threads started, main ends

thread1 going to sleep for 3519

thread2 going to sleep for 1689

thread3 going to sleep for 5565

thread2 finished

thread1 finished

thread3 finished

- 说明

- 由于线程3休眠时间最长，所以最后结束，线程2休眠时间最短，所以最先结束
- 每次运行，都会产生不同的随机休眠时间，所以结果都不相同

8.1.3 Runnable接口

- Runnable接口

- Java多线程机制的一个重要部分，实际上它只有一个run()方法
- Thread类实现了Runnable接口，相对于Thread类，它更适合于多个线程处理同一资源
- 实现Runnable接口的类的对象可以用来创建线程，这时start方法启动此线程就会在此线程上运行run()方法
- 在编写复杂程序时相关的类可能已经继承了某个基类，而Java不支持多继承，在这种情况下，便需要通过实现Runnable接口来生成多线程

8.1.3 Runnable接口(续)

——例8_3

- 使用**Runnable**接口实现例8_1功能

```
public class Ex8_3{  
    public static void main( String [] args ) {  
        System.out.println("main thread starts");  
        FactorialThread t=new FactorialThread(10);  
        new Thread(t).start();  
        System.out.println("new thread started,main thread ends " );  
    }  
}
```

8.1.3 Runnable接口(续)

——例8_3

```
class FactorialThread implements Runnable {
    private int num;
    public FactorialThread( int num ) {
        this.num=num;
    }
    public void run() {
        int i=num;
        int result=1;
        while(i>0) {
            result=result*i;
            i=i-1;
        }
        System.out.println("The factorial of "+num+" is "+result);
        System.out.println("new thread ends");
    }
}
```

8.1.3 Runnable接口(续)

——例8_4

- 使用Runnable接口实现例8_2功能

```
public class Ex8_4{
    public static void main( String [] args ) {
        TestThread thread1 = new TestThread();
        TestThread thread2 = new TestThread();
        TestThread thread3 = new TestThread();
        System.out.println( "Starting threads" );

        new Thread(thread1,"Thread1").start();
        new Thread(thread2,"Thread2").start();
        new Thread(thread3,"Thread3").start();

        System.out.println( "Threads started, main ends\n" );
    }
}
```

8.1.3 Runnable接口(续)

——例8_4

```
class TestThread implements Runnable {
    private int sleepTime;
    public TestThread() {
        sleepTime = ( int ) ( Math.random() * 6000 );
    }
    public void run() {
        try {
            System.out.println(
                Thread.currentThread().getName() + " going to sleep for "
                    + sleepTime );
            Thread.sleep( sleepTime );
        }
        catch ( InterruptedException exception ) {};
        System.out.println( Thread.currentThread().getName()+ "finished" );
    }
}
```

8.1.4 线程间的数据共享

- 代码共享
 - 多个线程的执行代码来自同一个类的run方法时，即称它们共享相同的代码
- 数据共享
 - 当共享访问相同的对象时，即它们共享相同的数据
 - 使用Runnable接口可以轻松实现多个线程共享相同数据，只要用同一个实现了Runnable接口的实例作为参数创建多个线程就可以了

8.1.4 线程间的数据共享(续)

——例8_5

- 修改例8_4，只用一个**Runnable**类型的对象为参数创建3个新线程。

```
public class Ex8_5 {  
    public static void main( String [] args ) {  
        TestThread threadobj = new TestThread();  
        System.out.println( "Starting threads" );  
  
        new Thread(threadobj,"Thread1").start();  
        new Thread(threadobj,"Thread2").start();  
        new Thread(threadobj,"Thread3").start();  
  
        System.out.println( "Threads started, main ends\n" );  
    }  
}
```

8.1.4 线程间的数据共享(续)

——例8_5

```
class TestThread implements Runnable {  
    private int sleepTime;  
    public TestThread() {  
        sleepTime = ( int ) ( Math.random() * 6000 );  
    }  
    public void run() {  
        try {  
            System.out.println(  
                Thread.currentThread().getName() + " going to sleep for " +  
                    sleepTime );  
            Thread.sleep( sleepTime );  
        }  
        catch ( InterruptedException exception ) {};  
        System.out.println( Thread.currentThread().getName() + "finished" );  
    }  
}
```


8.1.4 线程间的数据共享(续)

——例8_5运行结果

- 运行结果

Starting threads
Thread1 going to sleep for 966
Thread2 going to sleep for 966
Threads started, main ends
Thread3 going to sleep for 966
Thread1 finished
Thread2 finished
Thread3 finished

- 说明

- 因为是用一个Runnable类型对象创建的3个新线程，这三个线程就共享了这个对象的私有成员sleepTime，在本次运行中，三个线程都休眠了966毫秒

8.1.4 线程间的资源共享(续)

- 独立的同时运行的线程有时需要共享一些数据并且考虑到彼此的状态和动作
 - 例如生产/消费问题：生产线程产生数据流，然后这些数据流再被消费线程消费
 - 假设一个Java应用程序，其中有一个线程负责往文件写数据，另一个线程从同一个文件中读出读数据，因为涉及到同一个资源，这里是同一个文件，这两个线程必须保证某种方式的同步

8.1.4 线程间的数据共享(续)

——例8_6

- 用三个线程模拟三个售票口，总共出售**200**张票
 - 用3个线程模仿3个售票口的售票行为
 - 这3个线程应该共享200张票的数据

```
public class Ex8_6{  
    public static void main(String[] args){  
        SellTickets t=new SellTickets();  
        new Thread(t).start();  
        new Thread(t).start();  
        new Thread(t).start();  
    }  
}
```

8.1.4 线程间的数据共享(续)

——例8_6

```
class SellTickets implements Runnable{
    private int tickets=200;
    public void run() {
        while(tickets>0){
            System.out.println( Thread.currentThread().getName() +
                                " is selling ticket "+tickets--);
        }
    }
}
```

8.1.4 线程间的数据共享(续)

——例8_6运行结果

- 运行结果选最后几行如下
 - Thread-2 is selling ticket 6
 - Thread-1 is selling ticket 5
 - Thread-0 is selling ticket 4
 - Thread-2 is selling ticket 3
 - Thread-1 is selling ticket 2
 - Thread-0 is selling ticket 1
- 说明
 - 在这个例子中，创建了3个线程，每个线程调用的是同一个 **SellTickets** 对象中的 **run()** 方法，访问的是同一个对象中的变量 (**tickets**)
 - 如果是通过创建 **Thread** 类的子类来模拟售票过程，再创建3个新线程，则每个线程都会有各自的方法和变量，虽然方法是相同的，但变量却是各有**200张票**，因而结果将会是各卖出**200张票**，和原意就不符了

8.1.4 线程间的数据共享(续)

——例8_6_2

- 用三个线程模拟三个售票口，总共出售**200**张票

```
public class Ex8_6_2{  
    public static void main(String[] args){  
        SellTickets t1=new SellTickets();  
        SellTickets t2=new SellTickets();  
        SellTickets t3=new SellTickets();  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

8.1.4 线程间的数据共享(续)

——例8_6_2

```
class SellTickets extends Thread{
    private int tickets=200;
    public void run() {
        while(tickets>0){
            System.out.println( Thread.currentThread().getName() +
                                " is selling ticket "+tickets--);
        }
    }
}
```

8.1.5 多线程的同步控制

- 有时线程之间彼此不独立、需要同步
 - 线程间的互斥
 - 同时运行的几个线程需要共享一个（些）数据
 - 一个线程对共享的数据进行操作时，不允许其他线程打断它，否则会破坏数据的完整性。即被多个线程共享的数据，在某一时刻只允许一个线程对其进行操作
 - “生产者/消费者” 问题
 - 生产者产生数据，消费者消费数据，具体来说，假设有一个Java应用程序，其中有一个线程负责往数据区写数据，另一个线程从同一数据区中读数据，两个线程可以并行执行（类似于流水线上的两道工序）
 - 如果数据区已满，生产者要等消费者取走一些数据后才能再放；而当数据区没有数据时，消费者要等生产者放入一些数据后再取

8.1.5 多线程的同步控制(续)

——例8_7

- 用两个线程模拟存票、售票过程

- 假定开始售票处并没有票，一个线程往里存票，另外一个线程则往出卖票
- 我们新建一个票类对象，让存票和售票线程都访问它。本例采用两个线程共享同一个数据对象来实现对同一份数据的操作

```
public class Ex8_7 {  
    public static void main(String[] args) {  
        Tickets t=new Tickets(10);  
        new Consumer(t).start();  
        new Producer(t).start();  
    }  
}
```

8.1.5 多线程的同步控制(续)

——例8_7

```
class Tickets {  
    int number=0;           //票号  
    int size;               //总票数  
    boolean available=false; //表示目前是否有票可售  
    public Tickets(int size) { //构造函数，传入总票数参数  
  
        this.size=size;  
    }  
}
```

8.1.5 多线程的同步控制(续)

——例8_7

```
class Producer extends Thread{
    Tickets t=null;
    public Producer(Tickets t){
        this.t=t;
    }
    public void run(){
        while( t.number < t.size){
            System.out.println("Producer puts ticket "
                               +(++t.number));
            t.available=true;
        }
    }
}
```

8.1.5 多线程的同步控制(续)

——例8_7

多线程编程基础

```
class Consumer extends Thread { //售票线程

    Tickets t=null;
    int i=0;
    public Consumer(Tickets t){
        this.t=t;
    }
    public void run(){
        while(i<t.size) {
            if(t.available==true && i<=t.number)
                System.out.println("Consumer buys ticket "+(++i));
            if(i==t.number)
                t.available=false;
        }
    }
}
```

8.1.5 多线程的同步控制(续)

——例8_7运行结果

- 运行结果

```
Producer puts ticket 1
Producer puts ticket 2
Producer puts ticket 3
Producer puts ticket 4
Producer puts ticket 5
Producer puts ticket 6
Producer puts ticket 7
Producer puts ticket 8
Consumer buys ticket 1
Consumer buys ticket 2
Consumer buys ticket 3
Consumer buys ticket 4
Consumer buys ticket 5
Consumer buys ticket 6
Consumer buys ticket 7
Consumer buys ticket 8
Producer puts ticket 9
Producer puts ticket 10
Consumer buys ticket 9
Consumer buys ticket 10.
```

- 通过让两个线程操纵同一个票类对象，实现了数据共享的目的

8.1.5 多线程的同步控制(续)

——例8_7修改

- 设想一下，假如售票线程运行到**t.available=false**之前，CPU切换到存票线程，存票线程将**available**置为**true**，并直到整个存票线程结束。再次切换到售票线程后，售票线程执行**t.available=false**。此时售票号小于存票数，且存票线程已经结束不再能将**t.available**置为**true**，则售票线程陷入了死循环
- 如果我们在**t.available=false**之前加上**sleep**语句，让售票线程多停留一会，则可以更加清楚地看到这个问题

```
if(i==t.number) {  
    try{  
        Thread.sleep(1);  
    } catch ( InterruptedException exception ) {  
    }  
    t.available=false;  
}
```

8.1.5 多线程的同步控制(续)

——例8_7修改后运行结果

- 修改后运行结果

```
Producer puts ticket 1
Producer puts ticket 2
Producer puts ticket 3
Producer puts ticket 4
Producer puts ticket 5
Producer puts ticket 6
Producer puts ticket 7
Producer puts ticket 8
Consumer buys ticket 1
Consumer buys ticket 2
Consumer buys ticket 3
Consumer buys ticket 4
Consumer buys ticket 5
Consumer buys ticket 6
Consumer buys ticket 7
Consumer buys ticket 8
Producer puts ticket 9
Producer puts ticket 10
```

8.1.5 多线程的同步控制(续)

——解决例8_7的问题

- 如何避免上面这种意外，让我们的程序是“线程安全”的呢？
 - 解决线程的同步/互斥问题
 - 存票线程和售票线程应保持互斥关系。即售票线程执行时不进入存票线程、存票线程执行时不进入售票线程
- **Java** 使用的同步机制是监视器
 - 每个对象都只有一个“锁旗标”与之相连，利用多线程对其的争夺可实现线程间的互斥操作
 - 当线程A获得了一个对象的锁旗标后，线程B必须等待线程A完成规定的操作、并释放出锁旗标后，才能获得该对象的锁旗标，并执行线程B中的操作

8.1.5 多线程的同步控制(续)

——线程同步(Synchronization)

- 线程同步的概念，包括互斥和协作
 - 互斥：许多线程在同一个共享数据上操作而互不干扰，同一时刻只能有一个线程访问该共享数据。因此有些方法或程序段在同一时刻只能被一个线程执行，称之为监视区
 - 协作：多个线程可以有条件地同时操作共享数据。执行监视区代码的线程在条件满足的情况下可以允许其它线程进入监视区

8.1.5 多线程的同步控制(续)

——synchronized关键字

- **synchronized** ——线程同步关键字
 - 用于指定需要同步的代码段或方法，也就是监视区
 - 可实现与一个锁旗标的交互。例如：
 - **synchronized** (对象) { 代码段 }
 - **synchronized**的功能是：首先判断对象的锁旗标是否在，如果在就获得锁旗标，然后就可以执行紧随其后的代码段；如果对象的锁旗标不在（已被其他线程拿走），就进入等待状态，直到获得锁旗标
 - 当被**synchronized**限定的代码段执行完，就释放锁旗标

8.1.5 多线程的同步控制(续)

——synchronized关键字

- 将需要互斥的语句段放入**synchronized(object){}**语句框中, 且两处的**object**是相同的

```
class Producer extends Thread {
    Tickets t=null;
    public Producer(Tickets t) { this.t=t;}
    public void run() {
        while((t.number)<t.size) {
            synchronized(t) { // 申请对象t的锁旗标
                System.out.println("Producer puts ticket "+(++t.number));
                t.available=true;
            } // 释放对象t的锁旗标
        }
        System.out.println("Producer ends!");
    }
}
```

8.1.5 多线程的同步控制(续)

——synchronized关键字

```
class Consumer extends Thread {
    Tickets t=null;
    int i=0;
    public Consumer(Tickets t) { this.t=t; }
    public void run() {
        while(i<t.size) {
            synchronized(t) { //申请对象t的锁旗标
                if(t.available==true && i<=t.number)
                    System.out.println("Consumer buys ticket "+(++i));
                if(i==t.number) {
                    try{Thread.sleep(1);}catch(Exception e){}
                    t.available=false;
                }
            } //释放对象t的锁旗标
        }
        System.out.println("Consumer ends");
    }
}
```

8.1.5 多线程的同步控制(续)

——synchronized关键字

- 说明

- 存票程序段和售票程序段为获得同一对象的锁旗标而实现互斥操作
- 当线程执行到synchronized的时候，检查传入的实参对象，并申请得到该对象的锁旗标。如果得不到，那么线程就被放到一个与该对象锁旗标相对应的等待线程池中。直到该对象的锁旗标被归还，池中的等待线程才能重新去获得锁旗标，然后继续执行下去
- 除了可以对指定的代码段进行同步控制之外，还可以定义整个方法在同步控制下执行，只要在方法定义前加上synchronized关键字即可

8.1.5 多线程的同步控制(续)

——例8_8

- 实现例8_7功能。将互斥方法放在共享的资源类Tickets中

```
class Tickets {  
    int size; //票总数  
    int number=0; //存票序号  
    int i=0; //售票序号  
    boolean available=false; //是否有待售的票  
    public Tickets(int size) { this.size=size; }  
    public synchronized void put() { //同步方法，实现存票的功能  
        System.out.println("Producer puts ticket "++number);  
        available=true;  
    }  
    public synchronized void sell() { //同步方法，实现售票的功能  
        if(available==true && i<=number)  
            System.out.println("Consumer buys ticket "++i);  
        if(i==number) available=false;  
    }  
}
```

8.1.5 多线程的同步控制(续)

——例8_8说明

- 说明

- 同步方法使用的锁旗标关联对象正是方法所属的实例对象。在例8_8中，正是因为put和sell两个同步方法都属于同一个Tickets类的对象，所以实现了同步
- 由于要实现多线程的数据共享，即多个线程对同一数据资源进行操作，就可能造成一个线程对资源进行了部分处理，另一个线程就插进来对其进行处理，这样就会破坏共享数据的完整性。因此，需要使用线程同步与互斥技术，防止不同的线程同时对共享数据进行修改操作。
数据共享和线程互斥操作经常是密不可分的

8.1.6 线程之间的通信

- 为了更有效地协调不同线程的工作，需要在线程间建立沟通渠道，通过线程间的“对话”来解决线程间的同步问题
- `java.lang.Object` 类的一些方法为线程间的通讯提供了有效手段
 - `wait()` 如果当前状态不适合本线程执行，正在执行同步代码（`synchronized`）的某个线程A调用该方法（在对象x上），该线程暂停执行而进入对象x的等待池，并释放已获得的对象x的锁旗标。线程A要一直等到其他线程在对象x上调用`notify`或`notifyAll`方法，才能够在重新获得对象x的锁旗标后继续执行（从`wait`语句后继续执行）

8.1.6 线程之间的通信(续)

——notify()和notifyAll()方法

- notify() 随机唤醒一个等待的线程，本线程继续执行
 - 线程被唤醒以后，还要等发出唤醒消息者释放监视器，这期间关键数据仍可能被改变
 - 被唤醒的线程开始执行时，一定要判断当前状态是否适合自己运行
- notifyAll() 唤醒所有等待的线程，本线程继续执行

8.1.6 线程之间的通信(续)

——例8_9

- 修改例8_8，使每存入一张票，就售一张票，售出后，再存入

```
class Tickets {  
    .....  
    public synchronized void put() {  
        if(available) //如果还有存票待售，则存票线程等待  
            try{ wait();} catch(Exception e){}  
        System.out.println("Producer puts ticket "+(++number));  
        available=true;  
        notify(); //存票后唤醒售票线程开始售票  
    }  
    public synchronized void sell() {  
        if(!available) //如果没有存票，则售票线程等待  
            try{ wait();} catch(Exception e){}  
        System.out.println("Consumer buys ticket "+(number));  
        available=false;  
        notify(); //售票后唤醒存票线程开始存票  
        if (number==size) number=size+1 ; //在售完最后一张票后，  
            //设置一个结束标志， number>size表示售票结束  
    }  
}
```

8.1.6 线程之间的通信(续)

——例8_9

```
class Producer extends Thread {
    Tickets t=null;
    public Producer(Tickets t) {
        this.t=t;
    }
    public void run() {
        while(t.number<t.size)  t.put();
    }
}

class Consumer extends Thread {
    Tickets t=null;
    public Consumer(Tickets t) { this.t=t; }
    public void run() {
        while(t.number<=t.size)  t.sell();
    }
}
```

8.1.6 线程之间的通信(续)

——例8_9运行结果

- 运行结果

Producer puts ticket 1
Consumer buys ticket 1
Producer puts ticket 2
Consumer buys ticket 2
Producer puts ticket 3
Consumer buys ticket 3
Producer puts ticket 4
Consumer buys ticket 4
Producer puts ticket 5
Consumer buys ticket 5
Producer puts ticket 6
Consumer buys ticket 6
Producer puts ticket 7
Consumer buys ticket 7
Producer puts ticket 8
Consumer buys ticket 8
Producer puts ticket 9
Consumer buys ticket 9
Producer puts ticket 10
Consumer buys ticket 10

8.1.6 线程之间的通信(续)

——例8_9说明

- 程序说明

- 当Consumer线程售出票后，available值变为false，当Producer线程放入票后，available值变为true
- 只有available为true时，Consumer线程才能售票，否则就必须等待Producer线程放入新的票后的通知
- 只有available为false时，Producer线程才能放票，否则必须等待Consumer线程售出票后的通知
- 可见通过线程间的通信实现了我们的要求

8.1.7 后台线程

- 后台线程

- 也叫守护线程，通常是为了辅助其它线程而运行的线程
- 它不妨碍程序终止
- 一个进程中只要还有一个前台线程在运行，这个进程就不会结束；如果一个进程中的所有前台线程都已经结束，那么无论是否还有未结束的后台线程，这个进程都会结束
- “垃圾回收” 便是一个后台线程
- 如果对某个线程对象在启动（调用`start`方法）之前调用了`setDaemon(true)`方法，这个线程就变成了后台线程

8.1.7 后台线程(续)

——例8_10

- 创建一个无限循环的后台线程，验证主线程结束后，程序即结束

```
public class Ex8_10 {  
    public static void main(String[] args) {  
        ThreadTest t=new ThreadTest();  
        t.setDaemon(true);  
        t.start();  
    }  
}  
class ThreadTest extends Thread {  
    public void run() { while(true){} }  
}
```

- 运行程序，则发现整个程序在主线程结束时就随之中止运行了，如果注释掉**t.setDaemon(true)**语句，则程序永远不会结束

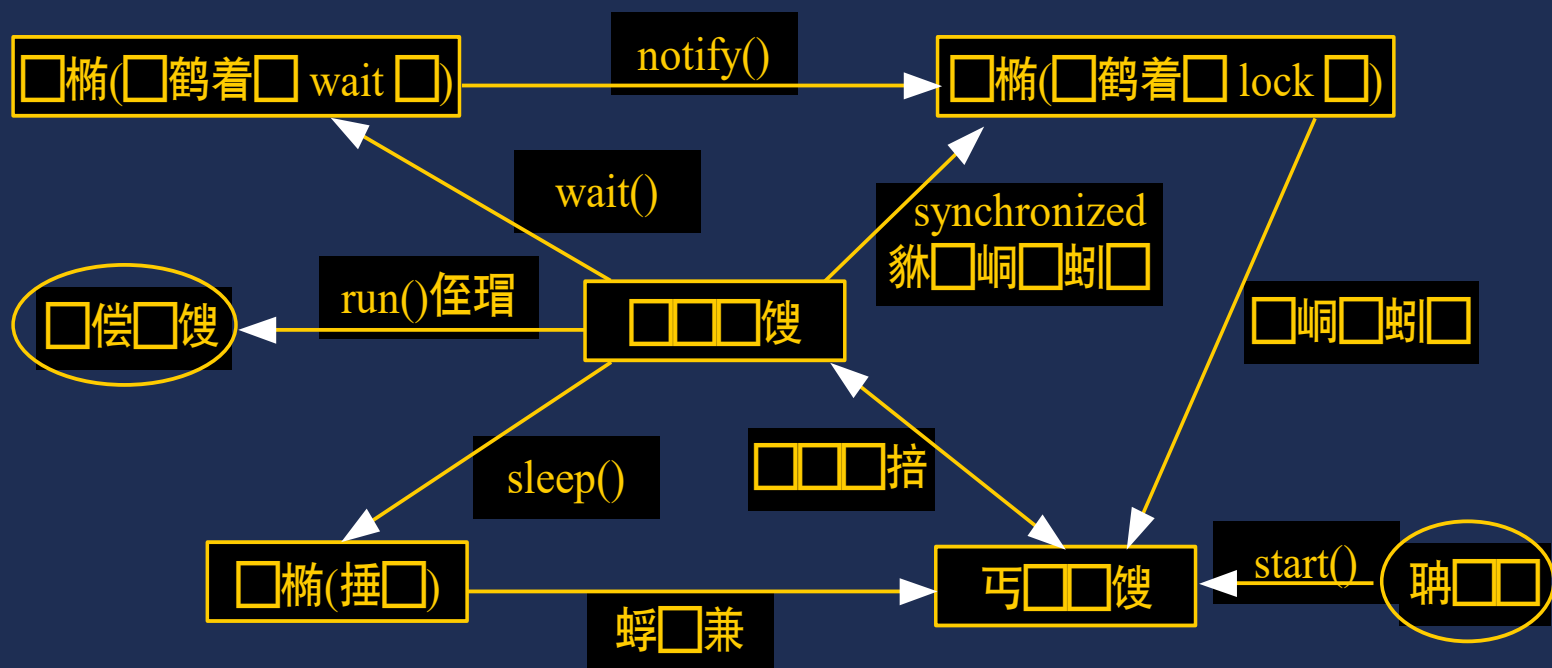
8.2 线程的生命周期

- 线程的生命周期
 - 线程从产生到消亡的过程
 - 一个线程在任何时刻都处于某种线程状态
(thread state)

8.2.1 线程的几种基本状态

- 线程生命周期状态图

线程的生命周期



8.2.1 线程的几种基本状态(续)

- 诞生状态
 - 线程刚刚被创建
- 就绪状态
 - 线程的 `start` 方法已被执行
 - 线程已准备好运行
- 运行状态
 - 处理机分配给了线程，线程正在运行
- 阻塞状态 (**Blocked**)
 - 在线程发出输入/输出请求且必须等待其返回
 - 遇到用 `synchronized` 标记的方法而未获得其监视器暂时不能进入执行时
- 休眠状态 (**Sleeping**)
 - 执行 `sleep` 方法而进入休眠
- 死亡状态
 - 线程已完成或退出

8.2.2 死锁问题

- 死锁

- 线程在运行过程中，其中某个步骤往往需要满足一些条件才能继续进行下去，如果这个条件不能满足，线程将在这个步骤上出现阻塞
- 线程A可能会陷于对线程B的等待，而线程B同样陷于对线程C的等待，依次类推，整个等待链最后又可能回到线程A。如此一来便陷入一个彼此等待的轮回中，任何线程都动弹不得，此即所谓死锁（**deadlock**）
- 对于死锁问题，关键不在于出现问题后调试，而是在于预防

8.2.3 控制线程的生命

- 结束线程的生命
 - 用stop方法可以结束线程的生命
 - 但如果一个线程正在操作共享数据段，操作过程没有完成就用stop结束的话，将会导致数据的不完整，因此并不提倡使用此方法
 - 通常，可通过控制run方法中循环条件的方式来结束一个线程

8.2.3 控制线程的生命(续)

——例8_12

- 线程不断显示递增整数，按下回车键则停止执行

```
import java.io.*;
public class Ex8_12{
    public static void main(String[] args) throws IOException{
        TestThread t=new TestThread();
        t.start();
        new BufferedReader(new InputStreamReader(System.in))
            .readLine(); //等待键盘输入
        t.stopme(); //调用stopme方法结束t线程
    }
}
```

8.2.3 控制线程的生命(续)

——例8_12

线程的生命周期

```
class TestThread extends Thread{
    private boolean flag=true;
    public void stopme() { //在此方法中控制循环条件
        flag=false;
    }
    public void run() {
        int i=0;
        while(flag) {
            System.out.println(i++); //如果flag为真则一直显示递增整数
        }
    }
}
```

- 运行效果为按下回车键后则停止显示

8.3 线程的优先级

- 线程调度

- 在单CPU的系统中，多个线程需要共享CPU，在任何时间点上实际只能有一个线程在运行
- 控制多个线程在同一个CPU上以某种顺序运行称为线程调度
- Java虚拟机支持一种非常简单的、确定的调度算法，叫做固定优先级算法。这个算法基于线程的优先级对其进行调度

8.3 线程的优先级(续)

- 线程的优先级

- 每个Java线程都有一个优先级，其范围都在1和10之间。默认情况下，每个线程的优先级都设置为5
- 在线程A运行过程中创建的新的线程对象B，初始状态具有和线程A相同的优先级
- 如果A是个后台线程，则B也是个后台线程
- 可在线程创建之后的任何时候，通过setPriority(int priority)方法改变其原来的优先级

线程的优先级

8.3 线程的优先级(续)

线程的优先级

- 基于线程优先级的线程调度
 - 具有较高优先级的线程比优先级较低的线程优先执行
 - 对具有相同优先级的线程，**Java**的处理是随机的
 - 底层操作系统支持的优先级可能要少于**10**个，这样会造成一些混乱。因此，只能将优先级作为一种很粗略的工具使用。最后的控制可以通过明智地使用**yield()**函数来完成
 - 我们只能基于效率的考虑来使用线程优先级，而不能依靠线程优先级来保证算法的正确性

8.3 线程的优先级(续)

- 假设某线程正在运行，则只有出现以下情况之一，才会使其暂停运行
 - 一个具有更高优先级的线程变为就绪状态（Ready）；
 - 由于输入/输出（或其他一些原因）、调用sleep、wait、yield方法使其发生阻塞；
 - 对于支持时间分片的系统，时间片的时间期满

8.3 线程的优先级(续)

——例8_13

- 创建两个具有不同优先级的线程，都从1递增到400000，每增加50000显示一次

```
public class Ex8_13{  
    public static void main(String[] args) {  
        TestThread[] runners = new TestThread[2];  
        for (int i = 0; i < 2; i++) runners[i] = new TestThread(i);  
        runners[0].setPriority(2); //设置第一个线程优先级为2  
        runners[1].setPriority(3); //设置第二个线程优先级为3  
        for (int i = 0; i < 2; i++) runners[i].start();  
    }  
}
```

线程的优先级

8.3 线程的优先级(续)

——例8_13

线程的优先级

```
class TestThread extends Thread{
    private int tick = 1;
    private int num;
    public TestThread(int i) { this.num=i; }
    public void run() {
        while (tick < 400000) {
            tick++;
            if ((tick % 50000) == 0) { //每隔50000进行显示
                System.out.println("Thread #" + num + ", tick = " + tick);
                yield(); //放弃执行权
            }
        }
    }
}
```

8.3 线程的优先级(续)

——例8_13运行结果

线程的优先级

- 运行结果

```
Thread #1, tick = 50000
Thread #1, tick = 100000
Thread #1, tick = 150000
Thread #1, tick = 200000
Thread #1, tick = 250000
Thread #1, tick = 300000
Thread #1, tick = 350000
Thread #1, tick = 400000
Thread #0, tick = 50000
Thread #0, tick = 100000
Thread #0, tick = 150000
Thread #0, tick = 200000
Thread #0, tick = 250000
Thread #0, tick = 300000
Thread #0, tick = 350000
Thread #0, tick = 400000
```

- 结果说明

- 具有较高优先级的线程1一直运行到结束，具有较低优先级的线程0才开始运行
- 虽然具有较高优先级的线程1调用了yield方法放弃CPU资源，允许线程0进行争夺，但马上又被线程1抢夺了回去，所以有没有yield方法都没什么区别

8.3 线程的优先级(续)

——例8_13修改

- 如果在**yield**方法后增加一行**sleep**语句，让线程1暂时放弃一下在**CPU**上的运行，哪怕是1毫秒，则线程0也可以有机会被调度。修改后的**run**方法如下

```
public void run() {  
    while (tick < 400000) {  
        tick++;  
        if ((tick % 50000) == 0) {  
            System.out.println("Thread #" + num + ", tick = " + tick);  
            yield();  
            try{ sleep(1);}catch(Exception e){};  
        }  
    }  
}
```

线程的优先级

8.3 线程的优先级(续)

——例8_13修改后运行结果

线程的优先级

- 运行结果

```
Thread #1, tick = 50000
Thread #1, tick = 100000
Thread #1, tick = 150000
Thread #1, tick = 200000
Thread #0, tick = 50000
Thread #1, tick = 250000
Thread #1, tick = 300000
Thread #0, tick = 100000
Thread #1, tick = 350000
Thread #1, tick = 400000
Thread #0, tick = 150000
Thread #0, tick = 200000
Thread #0, tick = 250000
Thread #0, tick = 300000
Thread #0, tick = 350000
Thread #0, tick = 400000
```

- 说明

- 具有较低优先权的线程0在线程1没有执行完毕前也获得了一部分执行，但线程1还是优先完成了执行
- **Java**虚拟机本身并不支持某个线程抢夺另一个正在执行的具有同等优先级线程的执行权
- 通常，我们在一个线程内部插入`yield()`语句，这个方法会使正在运行的线程暂时放弃执行，这时具有同样优先级的线程就有机会获得调度开始运行，但较低优先级的线程仍将被忽略不参加调度

8.4 本章小结

- 本章内容
 - 线程的基础知识
 - 线程的生命周期
 - 线程的优先级
- 本章要求
 - 了解线程的概念
 - 学会如何通过**Thread**类和**Runnable**接口创建线程，如何实现多线程的资源共享和通信，以及如何控制线程的生命
 - 掌握线程同步的方法
 - 理解线程优先级的概念，以及基于优先级的线程调度